

Python Basics with NumPy

A Beginner's Companion Guide

Understanding the code, the concepts, and why they matter for deep learning

About This Guide

What You Just Built, and Why It Is the Bedrock of Everything

A plain-language walkthrough of the seven functions in the assignment

This assignment looks modest: a handful of short functions, mostly one or two lines each. But do not let the size fool you. Every single piece you wrote here is a building block that reappears, unchanged in spirit, inside real neural networks — including the ones running in industry today. This guide walks through each function in plain terms, defines the key concepts behind it, and shows exactly where that idea lives in the larger world of deep learning, from the foundations up to advanced topics.

The assignment quietly teaches three big lessons at once. The first is **how to compute with NumPy** instead of plain Python. The second is **vectorization** — operating on whole arrays at once rather than looping element by element. The third is a first look at the actual **mathematical machinery** of neural networks: activation functions, their derivatives, and loss functions. Keep those three threads in mind as you read; everything below is a variation on them.

HOW TO READ THIS GUIDE

Each section follows the same rhythm: the code you wrote, what it does in everyday language, the key concept defined, and a short note on *where this shows up in deep learning* — both the foundations and the advanced topics it leads to. You do not need any mathematics beyond multiplication and the idea of a square root.

1. Why NumPy and Not Plain Python?

The assignment begins by asking you to write the sigmoid function twice — first with Python's built-in `math` library, then with NumPy. This is deliberate. The first version reveals a problem; the second solves it.

```
# First version - uses math.exp(), works on ONE number only
import math
def basic_sigmoid(x):
    s = 1 / (1 + math.exp(-x))
    return s
```

This works for a single number. But the moment you try to feed it a list of numbers, such as `[1, 2, 3]`, it crashes — `math.exp()` simply does not understand a list. That failure is the whole point. In

deep learning you never work with one number at a time; you work with thousands or millions of them at once. So you need tools built for that.

KEY CONCEPT: THE NUMPY ARRAY

A NumPy **array** is a container that holds many numbers and lets you do math on all of them in a single operation. Where `math.exp()` handles one value, `np.exp()` applies the exponential to every value in an array at once. Arrays are the fundamental data structure of all scientific computing in Python — and, as you will see in the book, the direct ancestor of the *tensor*, which is what PyTorch and every deep learning framework use to hold data.

Two small demonstrations in the assignment make the magic concrete. `np.exp(np.array([1, 2, 3]))` returns three exponentials at once, and `np.array([1, 2, 3]) + 3` adds 3 to every element. Neither needs a loop. This effortless "do it to the whole array" behavior is the foundation of everything that follows.

WHERE THIS MAPS IN DEEP LEARNING

Foundation: arrays are how a network holds its inputs, its weights, and its outputs. A batch of images, a layer's worth of activations, a set of predictions — all are arrays. **Advanced:** the same idea scales up to *tensors* on a GPU, where billions of these element-wise operations happen in parallel. Learning to think in arrays now is learning to think the way every deep learning system works.

2. The Sigmoid Function

The NumPy version of sigmoid is the same formula, now able to handle a whole array:

```
def sigmoid(x):  
    s = 1 / (1 + np.exp(-x))  
    return s
```

Feeding it `[1, 2, 3]` now returns three sigmoid values at once: roughly `[0.73, 0.88, 0.95]`. The formula is the well-known logistic curve:

$$\text{sigmoid}(x) = 1 / (1 + e^{-x})$$

KEY CONCEPT: ACTIVATION FUNCTION

An **activation function** is a small nonlinear function applied to a neuron's output. Sigmoid takes any number — from minus a million to plus a million — and squashes it into the range between 0 and 1.

Because the result always sits between 0 and 1, it can be read as a *probability*. Inputs near zero come out near 0.5; large positive inputs approach 1; large negative inputs approach 0.

The word that matters most there is **nonlinear**. If a network used no activation functions — or only straight-line ones — then stacking many layers would be pointless, because a chain of straight lines is still just a straight line. The bend in the sigmoid curve is what lets a network learn complicated, curved relationships rather than only straight ones.

WHERE THIS MAPS IN DEEP LEARNING

Foundation: sigmoid is the classic output activation for *binary classification* — deciding between two options, like "cat or not a cat," where the output is the probability of one class. **Advanced:** the same sigmoid appears inside the gates of LSTMs and GRUs (the sequence models behind language and speech systems), where it acts as a soft on/off switch controlling what information to keep or forget. A function you wrote in one line is a core component of state-of-the-art architectures.

3. The Sigmoid Derivative (Gradient)

Next you computed the *slope* of the sigmoid at any point, in two tidy steps:

```
def sigmoid_derivative(x):  
    s = sigmoid(x)  
    ds = s * (1 - s)  
    return ds
```

Notice the elegance: the derivative is written entirely in terms of the sigmoid's own output, `s`. Once you know `s`, the slope is just `s × (1 - s)`. No exponentials to recompute.

KEY CONCEPT: THE GRADIENT

A **gradient** (also called the derivative or slope) answers one question: "if I nudge the input a little, how much does the output change, and in which direction?" It is the single most important quantity in training a neural network, because it tells each weight which way to move to reduce the error. Learning, in deep learning, is following gradients downhill.

Look at what the numbers reveal. The gradient is largest at the center (a maximum of just 0.25 at $x = 0$) and shrinks toward zero as inputs grow large in either direction. That shrinking is not a quirk — it is a famous limitation.

WHY THIS MATTERS: THE VANISHING-GRADIENT PROBLEM

Because the sigmoid's gradient is never larger than 0.25 and often much smaller, stacking many sigmoid layers means multiplying many tiny numbers together during training. The correction signal shrinks until it nearly vanishes before reaching the early layers, which then barely learn. This is the **vanishing-gradient problem**, and it is the single biggest reason the field largely moved from sigmoid to ReLU for hidden layers. You are seeing the root cause of a major historical turning point, right here in a two-line function.

WHERE THIS MAPS IN DEEP LEARNING

Foundation: gradients are the heart of *backpropagation*, the algorithm that teaches every neural network.
Advanced: the vanishing-gradient problem motivated entire inventions — ReLU activations, careful weight initialization, batch normalization, and the "skip connections" in residual networks (ResNets) that made very deep image models possible. This little derivative is the doorway to all of it.

4. Reshaping an Image into a Vector

The `image2vector` function takes an image — stored as a three-dimensional block of numbers — and flattens it into a single tall column:

```
def image2vector(image):  
    v = image.reshape(image.shape[0] * image.shape[1] * image.shape[2], 1)  
    return v
```

An image has a shape like (length, height, depth) — for a color photo, depth is 3, one layer each for red, green, and blue. This function unrolls all those numbers into one long column of (total values, 1). Crucially, it does not hardcode the size; it reads the dimensions from the image itself with `.shape`, so the same function works for any image.

KEY CONCEPT: SHAPE AND RESHAPING

Every array has a **shape** — the list of its dimensions, such as (3, 3, 2). **Reshaping** rearranges those same numbers into a different shape without changing the numbers themselves or how many there are. The only rule is that the total count must stay the same: a (3, 3, 2) image has 18 values, so it can become an (18, 1) column but never a (20, 1) one.

WHERE THIS MAPS IN DEEP LEARNING

Foundation: a fully connected network cannot read a 2D image grid directly; the image must first be flattened into a vector of numbers, exactly as done here. This is the precise step used in a first image classifier, such as recognizing handwritten digits. **Advanced:** flattening throws away the spatial layout — the network no longer knows which pixels were neighbors. Fixing that loss is the entire motivation for *convolutional neural networks* (CNNs), which keep the 2D structure and now power computer vision. So this function also quietly shows you the limitation that an advanced topic was invented to solve.

5. Normalizing Rows

This function rescales each row of a matrix so that every row has a "length" of exactly 1:

```
def normalize_rows(x):
    x_norm = np.linalg.norm(x, ord=2, axis=1, keepdims=True)
    x = x / x_norm
    return x
```

The first line measures the length of each row (its *norm*), and the second divides each row by its own length. The two small settings do important work: `axis=1` says "measure across each row," and `keepdims=True` keeps the result shaped as a column so the division lines up correctly.

KEY CONCEPT: NORMALIZATION AND THE NORM

The **norm** of a row is its length, found the same way as the hypotenuse of a triangle: square every value, add them, take the square root. **Normalization** means rescaling data to a standard size — here, making every row unit length. The values keep their *direction* (their relative proportions) but lose differences in raw *magnitude*.

KEY CONCEPT: BROADCASTING

When you divide a (2, 3) matrix by a (2, 1) column, NumPy automatically stretches the column across all three positions of each row so the shapes line up. This automatic stretching is called **broadcasting**. The rule: two dimensions fit together when they are equal or when one of them is 1. The size-1 dimension is the one that stretches. Broadcasting is what lets you write clean array math without writing loops, and `keepdims=True` exists precisely to make broadcasting line up correctly.

WHERE THIS MAPS IN DEEP LEARNING

Foundation: normalizing inputs so features sit on a comparable scale helps a network train faster and more reliably — the same idea as scaling features before training. **Advanced:** normalization grew into one of the most important tools in modern deep learning: *batch normalization* and *layer normalization*, which normalize values *inside* the network between layers, are standard in nearly every large model, including the transformers behind today's language systems. Broadcasting, meanwhile, is happening invisibly in every line of neural-network math you will ever write.

6. The Softmax Function

Softmax turns a row of raw scores into a row of probabilities that add up to 1:

```
def softmax(x):  
    x_exp = np.exp(x)                    # make every score positive  
    x_sum = np.sum(x_exp, axis=1, keepdims=True) # total per row  
    s = x_exp / x_sum                    # each score / its row total  
    return s
```

The three steps are: exponentiate every score (which makes them all positive and amplifies the differences), add up each row, then divide each score by its row's total. The result is a set of probabilities — all between 0 and 1, summing to 1 — so they can be read as "the chance of each option." Notice `axis=1` and `keepdims=True` appear again, doing the same jobs as before.

KEY CONCEPT: SOFTMAX AND PROBABILITY DISTRIBUTIONS

Where sigmoid gives a single probability for a yes/no choice, **softmax** spreads probability across *many* options that compete with one another. Because the outputs are forced to sum to 1, giving more probability to one option necessarily takes it from the others. This makes softmax the natural choice whenever a network must pick one answer from several.

WHERE THIS MAPS IN DEEP LEARNING

Foundation: softmax is the standard output activation for *multi-class classification* — choosing one digit out of ten, one object out of a thousand. In a handwritten-digit recognizer, softmax turns ten raw scores into ten probabilities, and the largest wins. **Advanced:** softmax is also the beating heart of the *attention mechanism* inside transformers, where it decides how much each word should "pay attention" to every other word. The same three lines you wrote are computed billions of times inside a modern language model.

7. Vectorization: The Big Idea

The assignment ends by comparing two ways to do the same arithmetic: the slow way with Python `for` loops, and the fast way with NumPy functions like `np.dot`.

```
# The slow way - a Python loop adds up products one at a time
dot = 0
for i in range(len(x1)):
    dot += x1[i] * x2[i]

# The fast way - NumPy does the whole thing in one call
dot = np.dot(x1, x2)
```

Both compute the same answer. But the NumPy version is shorter, clearer, and dramatically faster — often hundreds of times faster on large data. The reason is that NumPy runs the loop inside fast, optimized low-level code rather than slow line-by-line Python.

KEY CONCEPT: VECTORIZATION

Vectorization means replacing explicit loops with single operations that act on entire arrays at once. Beyond being tidier, it lets the computer process many numbers in parallel. This is not a minor convenience — it is the difference between a model that trains in hours and one that would take months.

WHERE THIS MAPS IN DEEP LEARNING

Foundation: forward propagation — pushing a whole batch of examples through a network — is written as vectorized matrix math precisely so it runs fast. Every `np.dot` you saw is a miniature version of the matrix multiplications a network performs millions of times. **Advanced:** vectorization is exactly what GPUs are built to accelerate; the entire deep learning hardware industry exists to do these array operations in massive parallel. Understanding vectorization is understanding *why* deep learning became practical at all.

8. Loss Functions: L1 and L2

Finally, you wrote two ways to measure how wrong a prediction is, comparing predictions `yhat` against true answers `y`:

```
# L1 - sum of absolute differences
def L1(yhat, y):
    loss = np.sum(np.abs(y - yhat))
```

```
    return loss

# L2 - sum of squared differences
def L2(yhat, y):
    loss = np.sum((y - yhat) ** 2)
    return loss
```

L1 adds up the plain distances between predictions and truth. L2 adds up the *squared* distances. Both are zero when the predictions are perfect and grow as the predictions get worse.

KEY CONCEPT: THE LOSS FUNCTION

A **loss function** is the single number that measures how badly a network is doing right now. Training has exactly one objective: make this number as small as possible. The loss is what gradients are computed against — it is the "downhill" that learning rolls toward. Without a loss function, a network would have no way to know whether it was improving.

The difference between L1 and L2 is how harshly they punish big mistakes. Because L2 squares the errors, one large miss dominates the total far more than under L1. That makes L2 very sensitive to outliers and is why squared error is the usual choice for predicting continuous numbers.

WHERE THIS MAPS IN DEEP LEARNING

Foundation: L2 loss is the basis of *Mean Squared Error*, the standard loss for *regression* — predicting a continuous number like a house price. (Its cousin for classification is cross-entropy, built on the logarithms hinted at by sigmoid.) **Advanced:** choosing and designing the right loss function is one of the most powerful levers a practitioner has. Specialized losses drive everything from object detection to image generation. Every one of them plays the same role you just implemented: a single number that says "here is how wrong we are," which the whole network then works to shrink.

Putting It All Together

Step back and look at what these small functions add up to. Without realizing it, you have already built, by hand, the core ingredients of a neural network:

What you wrote	What it really is	Where it leads
<code>np.exp</code> , <code>arrays</code>	The array, the basic container for all data	Tensors, GPUs, every framework
<code>sigmoid</code>	An activation function (nonlinearity)	Binary classification; LSTM/GRU gates
<code>sigmoid_derivative</code>	A gradient — the engine of learning	Backpropagation; the vanishing-gradient story
<code>image2vector</code>	Flattening an image into numbers	Image classifiers; motivates CNNs
<code>normalize_rows</code>	Normalization and broadcasting	Batch / layer norm in modern models
<code>softmax</code>	Turning scores into competing probabilities	Multi-class output; transformer attention
<code>vectorization</code>	Whole-array math instead of loops	Forward propagation; GPU acceleration
<code>L1</code> , <code>L2</code>	Loss — how wrong the network is	MSE for regression; all of training

WHY THIS ASSIGNMENT MATTERS

Every advanced topic in deep learning — convolutional networks, recurrent networks, transformers, the systems behind translation, image generation, and chat — is assembled from exactly these pieces: arrays operated on in a vectorized way, passed through nonlinear activations, measured by a loss, and improved by following gradients. Master these humble functions and you are not merely "warming up." You are learning the alphabet that every deep learning sentence is written in. The rest of the field is learning how to combine these letters into ever more powerful words.

If a single idea is worth carrying forward, let it be this: deep learning is not magic, and it is not out of reach. It is built from small, understandable steps like the ones in this assignment — combined cleverly, repeated at enormous scale, and pointed at interesting problems. You have just held every one of those steps in your hand.